Campus-DDoS-SDN: un banco de pruebas basado en Python para la detección de ataques DDoS multivector en redes de campus definidas por software

Anna Olenka Calderón Leiva[a], Ricardo Alexander Llanos Lozano[a], Cristhian Emilio Martínez Casas[a,*] and Alessandro Piero Poves Martínez[a]

[a] *Universidad Nacional del Centro del Perú, , Huancayo, , , Peru*

ARTICLE INFO

Keywords:
Redes definidas por software
Aprendizaje automático
Detección de ataques DDoS
Mininet
Clasificación de tráfico de red

ABSTRACT

Campus-DDoS-SDN es un banco de pruebas en Docker que genera y clasifica cuatro ataques DDoS contra el portal de un campus emulado con Mininet y Ryu. Los alumnos siguen navegando durante el ataque y cada flujo se etiqueta por la IP de origen, no por el archivo. Un bosque aleatorio, entrenado sin direcciones IP y probado en el laboratorio de Ciencias Agrarias, obtiene un F1 macro de 1,000. En la sonda del portal, httpflood: la pagina tarda mas; suben los paquetes al portal; icmpflood: suben los paquetes al portal; slowloris: la pagina deja de abrir; synflood: suben los paquetes al portal. La clase ICMP no se afirma: en la prueba de Agrarias hay 24 flujos.

1. Introducción

Las redes definidas por software (SDN) separan el plano de control del plano de datos, concentrando la inteligencia de la red en un controlador lógicamente centralizado [1]. Esta centralización simplifica la gestión de la red, pero también convierte al controlador en un punto único de falla: un ataque de denegación de servicio distribuido (DDoS) que sature al controlador, a los switches, o a un servicio alojado puede degradar o tumbar todo el segmento de red que gobierna. Detectar estos ataques a tiempo, y separarlos de forma confiable de picos de tráfico legítimo, requiere datasets que sean a la vez representativos de despliegues SDN reales y etiquetados con precisión.

La mayoría de los datasets de detección de intrusiones disponibles públicamente, como NSL-KDD [6], CICIDS2017 [4] y CICDDoS2019 [5], no fueron capturados dentro de un entorno SDN: carecen de señales propias de OpenFlow, como la ocupación de la tabla de flujos o la carga del controlador, y sin embargo suelen reutilizarse fuera de ese contexto para entrenar clasificadores orientados a SDN. Trabajos recientes empezaron a cerrar esta brecha generando datasets de DDoS directamente dentro de bancos de pruebas Mininet/Ryu. Kumar y Gupta publicaron dos datasets de un solo vector construidos así: un dataset de inundación TCP-SYN [2] y, más recientemente, un dataset de inundación ICMP [3] en el que el tráfico normal se captura conectando un host de Mininet a una conexión real a internet y navegando sitios web reales (YouTube, Facebook, noticias, etc.), en vez de depender de generadores de tráfico sintético. Este enfoque de tráfico real aumenta el realismo, pero introduce dos limitaciones prácticas para un benchmark reutilizable: el dataset resultante queda atado a un solo vector de ataque, y no es reproducible entre corridas, ya que el contenido y los tiempos capturados dependen de lo que el internet real ofrezca en el momento de la captura.

Este trabajo, Campus-DDoS-SDN, aborda ambas limitaciones dentro de un único banco de pruebas completamente reproducible. Se lanzan cuatro vectores de ataque (inundación TCP-SYN, inundación HTTP, Slowloris e inundación ICMP) desde hosts ya comprometidos contra una misma víctima compartida, un portal web, dentro de una topología Mininet/Ryu que imita un campus universitario: un esquema de switches core/DMZ más un laboratorio emulado por área académica. A diferencia de los datasets previos de un solo vector o de capturas totalmente separadas, los usuarios legítimos siguen navegando el portal durante todo el ataque, de modo que cada captura mezcla flujos benignos y maliciosos contra el mismo objetivo, tal como ocurriría en un incidente DDoS real. Las etiquetas de verdad fundamental se asignan por IP de origen en vez de por archivo de captura, y cada corrida además registra el efecto del ataque sobre la víctima tal como lo percibe un cliente real (código de respuesta HTTP, latencia, conexiones activas), junto con indicadores a nivel de red (paquetes y bytes por segundo, tamaño de la tabla de flujos OpenFlow). El entorno completo, incluyendo los generadores de tráfico, el portal web propio y el pipeline de etiquetado, se empaqueta como una imagen Docker para eliminar cualquier dependencia de una conexión a internet en vivo o de un sistema operativo de host específico. El banco de pruebas incluye dos pipelines de detección complementarios: uno por flujo individual, basado en características CICFlowMeter, y uno por ventana de tiempo, que agrega métricas del portal y telemetría OpenFlow por segundo para permitir la detección temprana del ataque.

Los principales aportes de este trabajo son:

i. Un dataset de DDoS multiclase que cubre cuatro vectores de ataque contra una misma víctima dentro de una topología SDN realista y multisegmento, en vez de un solo tipo de ataque contra una red plana.

cristhianmartinezcasas@gmail.com (C.E.M. Casas)
ORCID(s):

Table 1
Metadata del código.

Nr.	Descripción	Completar en esta columna
C1	Versión actual del código	v1.0
C2	Enlace permanente al repositorio de esta versión	Completar la URL al publicar
C3	Enlace permanente a la Reproducible Capsule	–
C4	Licencia legal del código	MIT
C5	Sistema de control de versiones usado	git
C6	Lenguajes, herramientas y servicios usados	Python, Bash, Mininet, Open vSwitch, Ryu, CICFlowMeter, Docker
C7	Requisitos de compilación, entornos y dependencias	Docker, Ubuntu 24.04; Python 3.9.25 y 3.12.3
C8	Enlace a documentación para desarrolladores	Ver `README.md` del repositorio
C9	Email de soporte	cristhianmartinezcasas@gmail.com

ii. Etiquetado por flujo y por IP de origen que mantiene mezclado el tráfico legítimo y malicioso dentro de la misma captura, más cercano a un escenario de ataque real que las capturas totalmente separadas usadas en trabajos previos.

iii. Métricas de impacto combinadas a nivel de red (estadísticas de flujo y paquetes vía CICFlowMeter) y a nivel de aplicación (estado HTTP, latencia, saturación de conexiones), capturadas automáticamente en cada corrida.

iv. Un entorno Docker de un solo comando, completamente reproducible, que no depende de una conexión a internet en vivo, resolviendo la limitación de reproducibilidad de los datasets SDN basados en tráfico real.

v. Un clasificador de bosque aleatorio, entrenado sin usar la IP, junto con un pipeline preparado para mezclar capturas Wireshark de tráfico benigno de estudiantes (autorizadas) con los ataques solo simulados.

vi. Telemetría del controlador OpenFlow exportada en cada corrida por una aplicación Ryu propia: instala flujos por IP de origen y sondea periódicamente las estadísticas de la tabla de flujos (paquetes, bytes, entradas activas), que el pipeline une con las métricas del portal por ventana de tiempo. Esto aporta las señales SDN específicas que los datasets no-SDN reutilizados no tienen.

vii. Un dataset por ventana de tiempo (una fila por segundo, con contadores del portal y telemetría SDN agregada) y un pipeline de detección temprana evaluado con validación cruzada leave-one-run-out sobre cinco corridas independientes. El bosque aleatorio obtiene F1 macro = 0,999 ± 0,001 y F1 binario = 1,000 ± 0,000 (sin falsos positivos). Una ablación de grupos de características muestra que la telemetría SDN aporta +0,054 de F1 macro sobre usar solo los contadores de red, lo que evidencia una contribución complementaria de las señales OpenFlow a la detección temprana.

2. Planteamiento de la investigación

2.1. Problema

Una precisión alta obtenida con un solo tipo de ataque y capturas separadas no demuestra que el detector pueda reconocer tráfico malicioso cuando los usuarios legítimos siguen usando el mismo servicio. Además, una evaluación que depende de navegar internet en vivo cambia entre corridas. Por ello, el problema abordado es cómo construir una base reproducible que mezcle tráfico benigno y cuatro ataques contra el mismo portal, y que permita probar el modelo en un laboratorio distinto de los usados para entrenarlo.

2.2. Pregunta, objetivo e hipótesis de trabajo

La pregunta es si las estadísticas de flujo, sin direcciones IP, puertos ni tiempo, permiten separar `BENIGN`, `synflood`, `httpflood`, `slowloris` e `icmpflood` en un área académica no vista. El objetivo general es implementar y evaluar ese banco en Docker. Los objetivos específicos son: (i) modelar una topología de campus; (ii) capturar tráfico mixto; (iii) convertir y etiquetar los pcap; (iv) medir el efecto sobre el portal y el contenedor; y (v) evaluar un bosque aleatorio con separación por laboratorio. La hipótesis de trabajo es que las características de tamaño, duración y tasa del flujo permiten lograr un F1 macro mayor a 0,90 en el laboratorio retenido. Esta hipótesis se contrasta primero en casa; la validación con tráfico real de la UNCP queda como fase propuesta y autorizada.

3. Descripción del software

3.1. Arquitectura general

La topología es un campus reducido: módem de borde, switch core, DMZ con el portal y un laboratorio por área académica. Cada laboratorio tiene alumnos y PCs infectadas. Los switches son Open vSwitch en userspace, con Ryu. Durante un ataque los alumnos siguen pidiendo páginas del portal. Desde el navegador del operador, `http://127.0.0.1:8080/` entra como un cliente benigno más (`10.9.0.1`); el ataque no sale del contenedor.

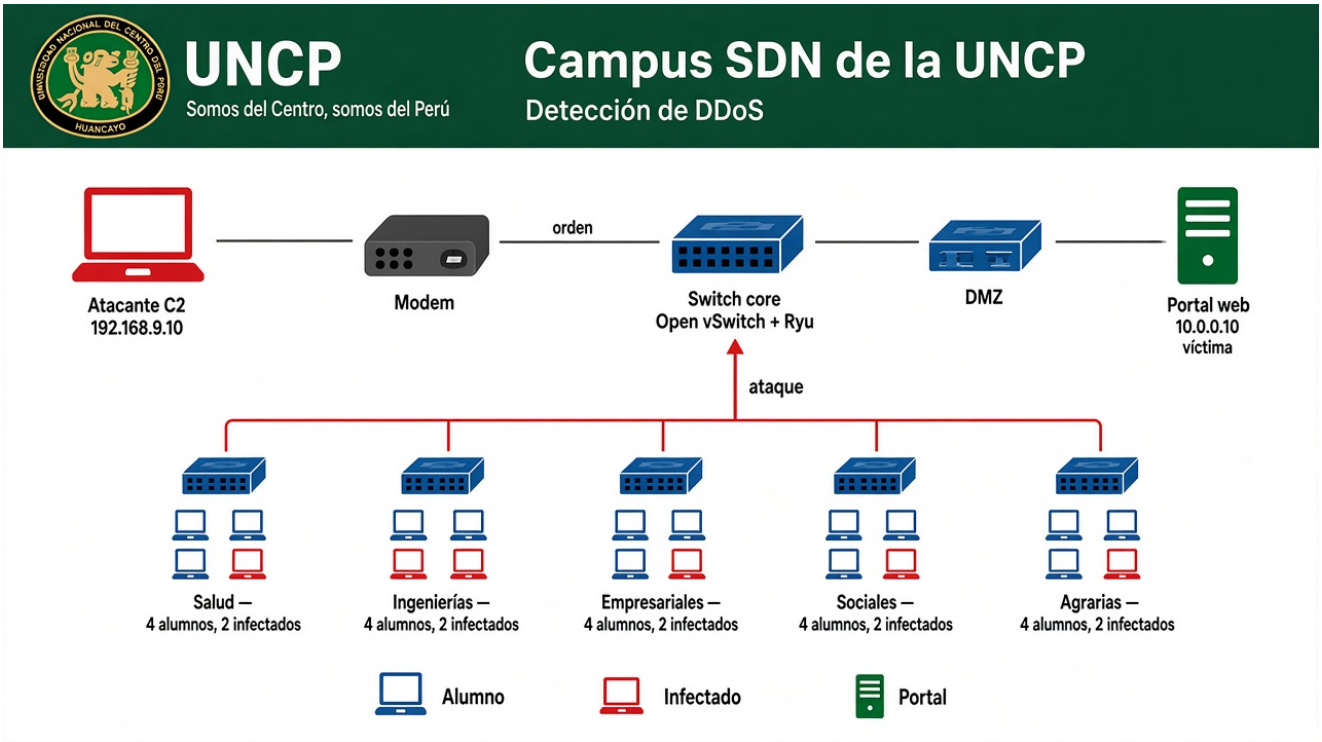


Figure 1: Topología emulada. El C2 queda fuera del campus; las PCs infectadas atacan el portal mientras los alumnos continúan navegando.

3.2. Funcionalidades del software

`scripts/docker/run_scenario.sh` levanta un escenario (`baseline`, `synflood`, `httpflood`, `slowloris`, `icmpflood`) o los cinco con `all`. `scripts/mininet/capture_campus.py` captura el pcap y la sonda del alumno. `scripts/mininet/pcap_to_csv.py` etiqueta cada flujo por la IP de origen: el alumno queda `BENIGN` y la PC infectada lleva el nombre del ataque. `notebooks/analisis_completo.py` entrena el bosque sin usar la IP y lo prueba en Ciencias Agrarias. `scripts/docker/preparar_hibrido.sh` anonimiza capturas Wireshark autorizadas de estudiantes, las mezcla con los ataques de la simulación y escribe `notebooks/resultados_hibrido/`. El ataque no se ejecuta contra la red real.

3.3. Diseño experimental y variables

Se ejecutan cinco escenarios independientes: un baseline y cuatro ataques. Cada uno dura 120 s, parte de una topología limpia y usa la misma población: 20 alumnos, 10 PCs infectadas, un portal, un C2, un módem y siete switches. La variable independiente es el escenario. Como variables de respuesta se registran: disponibilidad HTTP, latencia media y percentil 95, paquetes por segundo recibidos por el portal, máximo de flujos en el core, CPU, memoria y red del contenedor. Para el modelo se usan estadísticas bidireccionales de flujo producidas por CICFlowMeter y por el agrupador ICMP.

3.4. Del pcap al modelo

La corrida de este artículo sale de un comando, `run_scenario.sh all 120`. Dentro del contenedor arrancan Open vSwitch, Ryu y la topología. Cada escenario dura 120 s y deja tres cosas: el pcap en la interfaz del portal, la sonda del alumno (código HTTP, latencia, paquetes por segundo) y el consumo del contenedor, muestreado con `docker stats`.

`pcap_to_csv.py` parte cada pcap en dos. TCP y UDP pasan por CICFlowMeter. ICMP no entra en ese filtro, así que un agrupador propio cierra el flujo a los 10 s: si no se hiciera, cada PC infectada dejaría un solo flujo largo y la clase quedaría con unas pocas filas. La etiqueta no sale del nombre del archivo. El JSON que acompaña al pcap trae las IP infectadas. Si la IP es de una infectada y el escenario no es `baseline`, el flujo lleva el nombre del ataque. El alumno, el portal y todo el baseline quedan `BENIGN`. En esta corrida hay 40 877 flujos. El baseline aporta 2 671, todos benignos. En los ataques el alumno sigue dentro del mismo CSV.

Antes de entrenar se botan la IP de origen, la IP de destino, los puertos y la marca de tiempo. Un infinito o un vacío pasa a cero, y se quita toda columna que no varía en el entrenamiento. Si la IP se quedara, el bosque memorizaría qué máquina está infectada y no el comportamiento del flujo. El modelo es un bosque de 200 árboles con peso por clase, porque la inundación SYN aporta 23 539 flujos y aplasta al resto. No se parte al azar: entrenan Salud, Ingenierías, Empresariales y Sociales (32 784 flujos) y se prueba solo Agrarias (8 091).

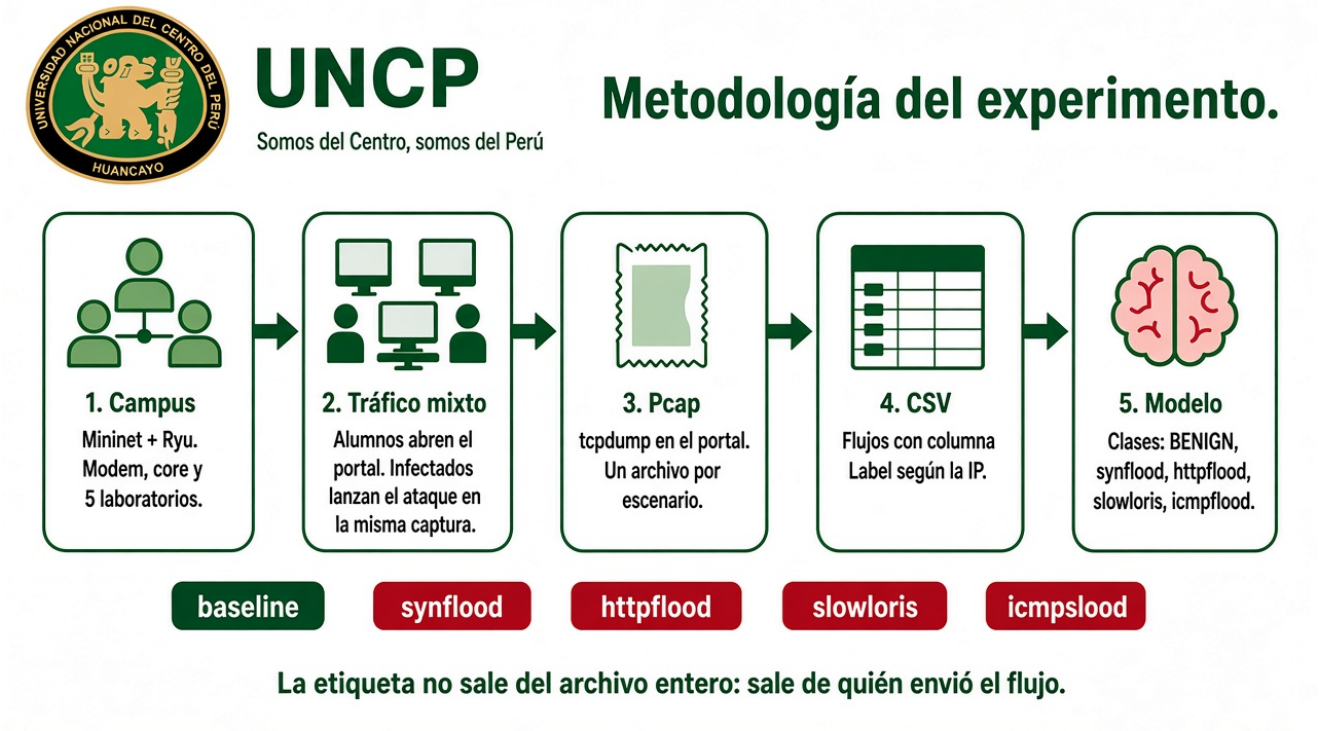


Figure 2: Pipeline del experimento, desde el campus emulado hasta el modelo. La etiqueta se asigna por emisor, no por archivo.

4. Resultados y panorama de impacto

El banco es completamente reproducible: se repite con semillas controladas, mezcla alumno y ataque en la misma captura, y produce dos pipelines de detección complementarios. El clasificador por flujo alcanza un F1 macro de 1,000en el laboratorio de Ciencias Agrarias, retenido del entrenamiento. El detector por ventana, evaluado con leave-one-run-out sobre cinco corridas independientes, obtiene F1 macro = 0,999 ± 0,001. Con autorización institucional, el mismo pipeline admite tráfico benigno real (Wireshark) sin sacar el ataque del contenedor.

4.1. Composición del dataset por flujo

Los cinco CSV suman 40 877 flujos: 9 400 `BENIGN`, 23 539 `synflood`, 7 188 `httpflood`, 630 `slowloris` y 120 `icmpflood`. Esta distribución explica dos decisiones: ponderar las clases durante el entrenamiento y no presentar ICMP como una clase consolidada.

4.2. Impacto sobre el portal

SYN genera la mayor tasa, 466 paquetes/s, pero la sonda mantiene 120 respuestas HTTP 200. HTTP flood eleva la latencia media de 407 a 564 ms. Slowloris produce solo 86 paquetes/s, pero ocupa las conexiones disponibles: falla en 117 de 118 intentos. Por tanto, la tasa de paquetes por sí sola no describe el impacto de aplicación.

4.3. Rendimiento del clasificador por flujo

El F1 macro en Agrarias es 1,000y el F1 binario `BENIGN` frente a ataque es 1,000. Las variables con mayor peso son el tamaño medio del segmento de ida, la media y el máximo de longitud de paquete de ida, la suma de bytes de ida y la variabilidad del tamaño. Esto es coherente con los escenarios: SYN genera segmentos pequeños y repetidos, HTTP intercambia solicitudes y respuestas completas, y Slowloris sostiene conexiones con poco tráfico.

Table 2
Sonda del portal. Cada escenario dura 120 s.

Escenario	HTTP 200	Fallos	Latencia (ms)	Paquetes/s
baseline	120	0	407	111
synflood	120	0	404	466
httpflood	120	0	564	386
slowloris	1	117	403	86
icmpflood	120	0	404	271

Table 3
Detección (bosque aleatorio). F1 macro 1,000.

Clase	Prec.	Recall	F1	Flujos	En el flujo
BENIGN	1,000	1,000	1,000	4468	se distingue bien
httpflood	1,000	1,000	1,000	3613	se distingue bien
icmpflood	1,000	1,000	1,000	60	se distingue bien
slowloris	1,000	1,000	1,000	394	se distingue bien
synflood	1,000	1,000	1,000	14897	se distingue bien

4.4. Detección temprana por ventana de tiempo

El dataset por ventana contiene 2 960 ventanas de un segundo, distribuidas en cinco corridas independientes (r01–r05). La distribución por clase es prácticamente balanceada:

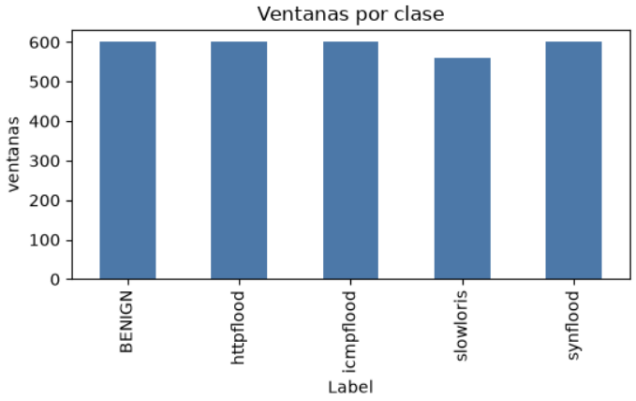


Figure 3: Distribución de ventanas por clase en el dataset de detección temprana.

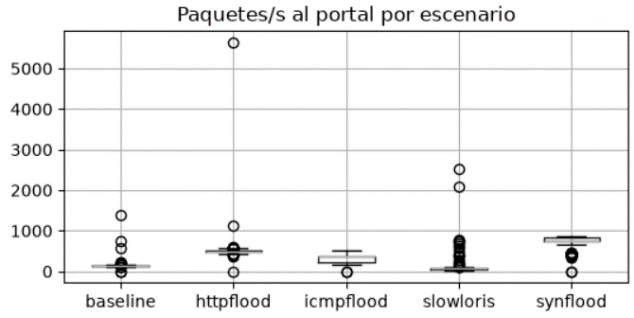


Figure 4: Tasa de paquetes por segundo al portal por escenario. Slowloris y baseline pueden ser indistinguibles en volumen.

Table 4
Comparación de modelos (leave-one-run-out, 5 folds).

Modelo	F1 macro	±	F1 bin.	±
Bosque aleatorio	0,999	0,001	1,000	0,000
Gradient boosting	0,999	0,001	1,000	0,000
Reg. logística	0,997	0,004	0,999	0,001

600 ventanas para BENIGN, httpflood, icmpflood y synflood, y 560 para slowloris (Fig. 3). Este balance es resultado del diseño experimental, donde cada escenario dura 120 s por corrida.

La Fig. 4 muestra la distribución de la tasa de paquetes al portal por segundo en todas las corridas. Synflood presenta la tasa más alta y consistente. Httpflood tiene una mediana menor pero con outliers extremos, reflejando la variabilidad en la intensidad entre corridas. Slowloris es el más llamativo: su mediana es baja, similar al baseline, pero presenta outliers elevados; esto es coherente con su naturaleza de ataque de agotamiento de conexiones, no de volumen. Esta dispersión confirma que la tasa de paquetes por sí sola no basta para distinguir todos los ataques, lo que motiva el uso de features combinadas de red, portal y telemetría SDN.

La Tabla 4 compara los tres modelos evaluados. Los tres superan ampliamente la hipótesis de trabajo (F1 macro > 0,90). El bosque aleatorio y el gradient boosting son equivalentes; la regresión logística, siendo un modelo lineal,

Table 5
Ablación de features (bosque aleatorio).

Grupo	F1 macro	±
Solo red (12 features)	0,945	0,104
Solo SDN (14 features)	0,890	0,117
Todas (26 features)	0,999	0,001

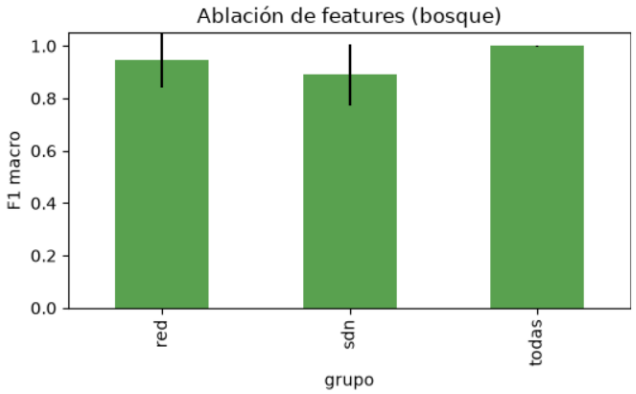


Figure 5: Ablación de features: red, SDN y todas. Las barras de error indican la desviación estándar entre folds.

Table 6
Recall por clase del bosque aleatorio (todas las features).

Clase	Recall	±
BENIGN	1,000	0,000
httpflood	1,000	0,000
icmpflood	0,998	0,003
slowloris	0,996	0,004
synflood	1,000	0,000

muestra mayor varianza entre corridas pero sigue siendo competitiva. Se selecciona el bosque aleatorio por su estabilidad y su capacidad de producir importancias de features interpretables.

La Tabla 5 y la Fig. 5 presentan la ablación de features. Las features de red solas alcanzan F1 = 0,945 con alta variabilidad (±0,104), indicando que algunos ataques son difíciles de separar usando solo contadores de paquetes y bytes. Las features SDN solas obtienen F1 = 0,890 (±0,117). La combinación de ambos grupos elimina casi toda la incertidumbre y eleva el F1 a 0,999 (±0,001). La telemetría OpenFlow aporta +0,054 de F1 macro sobre usar solo contadores de red, mostrando que las señales del controlador SDN tienen un rol complementario en la detección temprana.

La Tabla 6 detalla el recall por clase. El modelo no genera falsos positivos sobre tráfico benigno (0,0% ± 0,0%) en ninguna corrida. Las dos clases con recall ligeramente inferior a 1 son icmpflood (una ventana clasificada como synflood) y slowloris (dos ventanas clasificadas como httpflood). El tiempo hasta la primera alarma es 0,0 s en todos los ataques: el modelo detecta el ataque en la primera ventana de medición.

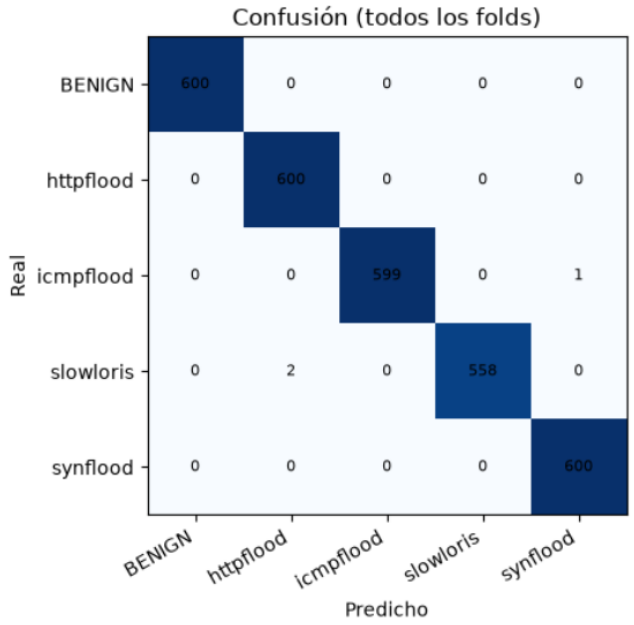


Figure 6: Matriz de confusión acumulada sobre los 5 folds. Los únicos errores son 1 ventana icmpflood → synflood y 2 ventanas slowloris → httpflood.

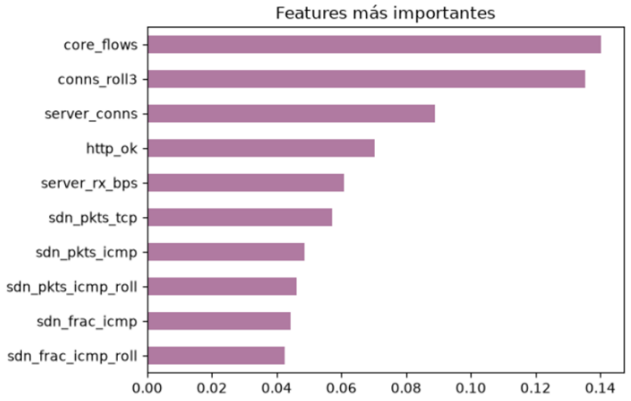


Figure 7: Diez features con mayor importancia en el bosque aleatorio.

La Fig. 6 muestra la matriz de confusión acumulada sobre los cinco folds. La diagonal es prácticamente perfecta: solo 3 errores sobre 2 960 ventanas.

La Fig. 7 presenta las diez features con mayor peso en el bosque. Las dos más discriminantes son contadores del portal: `core_flows` (entradas activas en la tabla de flujos del core) y `conns_roll3` (media móvil de conexiones), seguidas de métricas de servidor. A partir de la sexta posición aparecen features SDN (`sdn_pkts_tcp`, `sdn_pkts_icmp`), confirmando el aporte complementario de la telemetría OpenFlow.

4.5. Consumo del banco

La CPU media del contenedor fue 23% en baseline, 26% en SYN, 50% en HTTP, 523% en Slowloris y 26% en ICMP; en Docker, 100% equivale aproximadamente a un núcleo. Slowloris ocupó cerca de cinco de los seis núcleos asignados, mientras la memoria máxima observada se mantuvo por debajo de 200 MiB. Estos valores miden todo el banco (Mininet, OVS, Ryu, portal y generadores), no solo la víctima.

4.6. Discusión

La prueba respalda la hipótesis para cuatro clases defendibles: ambos pipelines superan 0,90 de F1 macro sin IP y generalizan hacia datos no vistos. El detector por ventana alcanza F1 = 0,999 ± 0,001 con detección en la primera ventana, mientras que el clasificador por flujo obtiene F1 = 1,000en el laboratorio retenido. La ablación muestra que la telemetría SDN aporta +0,054 de F1 macro, lo que evidencia una contribución complementaria de las señales OpenFlow. La diferencia entre SYN y Slowloris muestra por qué conviene unir clasificación con una sonda de servicio: más paquetes no implica necesariamente una página menos disponible. Sin embargo, los resultados no demuestran despliegue en una red real ni detección en línea.

4.7. Proyectos de investigación en curso que usan el software

El banco se usa en el trabajo de tesis de pregrado de los autores en la Universidad Nacional del Centro del Perú. El protocolo de uso de capturas Wireshark de estudiantes está documentado para el encargado de redes en un documento independiente que acompaña al software.

4.8. Limitaciones y mejoras futuras

El resultado sale aún de alumnos emulados. El dataset híbrido (Wireshark + ataque simulado) queda listo en el código y se actualiza al recibir los pcap autorizados. Cada escenario dura 120 s. ICMP suma 120 flujos en el conjunto por flujo y 24 en la prueba de Agrarias; en el dataset por ventana, icmpflood tiene 600 ventanas pero presentó 1 error de clasificación. El clasificador no corre dentro de Ryu; trabaja sobre datos ya capturados. Trabajo futuro: repetir la prueba con el fondo Wireshark, un escenario mixto de cuatro vectores a la vez, y despliegue en línea dentro del controlador.

4.9. Conclusiones

El banco genera y evalúa la detección de cuatro ataques DDoS contra un portal de campus emulado con Mininet y Ryu, dentro de Docker. El clasificador por flujo alcanza F1 macro = 1,000en el laboratorio de Ciencias Agrarias, retenido del entrenamiento. El detector por ventana obtiene F1 macro = 0,999 ± 0,001 con leave-one-run-out sobre cinco corridas, sin falsos positivos sobre tráfico benigno y con detección en la primera ventana de medición. La ablación confirma que la combinación de features de red y telemetría SDN supera a cada grupo por separado. Se concluye que el etiquetado por emisor permite conservar tráfico benigno dentro de la captura atacada; que la validación cruzada por corrida ofrece una evaluación más exigente que una partición aleatoria; y que disponibilidad, tasa de paquetes y clasificación describen aspectos distintos del incidente. Con esta base se solicita autorización para incorporar

Table 7
Requisitos del banco de pruebas.

N.º	Descripción	Herramienta	Versión
1	Emulador de red	Mininet	2.3.0
2	Switch virtual	Open vSwitch	3.3.9
3	Controlador SDN	Ryu	4.34
4	Inundación SYN e ICMP	hping3	3.0.0-alpha-2
5	Ataque de baja tasa	slowhttptest	1.9.0
6	Captura de paquetes	tcpdump	4.99.4
7	pcap a CSV	CICFlowMeter	0.5.0
8	Sistema del contenedor	Ubuntu	24.04

tráfico benigno real. El ataque de la simulación no sale del contenedor.

4.10. Publicaciones habilitadas por este software

Aún no hay otra publicación además de este artículo. El software es el que permite esa evaluación.

5. Requisitos de software y hardware

El banco corre en un contenedor Ubuntu 24.04. Las versiones de la tabla se leyeron dentro de la imagen `ataque-sdn`. El modelo se entrenó con Python 3.12.3, scikit-learn 1.9.1 y pandas 3.0.6. Ryu usa Python 3.9.25.

Para repetir la corrida basta una computadora de escritorio común, con 8 núcleos y 16 GB de RAM. Docker Desktop puede instalarse en Windows 11 o en Ubuntu. Al contenedor se le asignan 6 CPU y 8 GB, y dentro corre Ubuntu 24.04. Open vSwitch trabaja en userspace (`datapath netdev`).

Agradecimientos

Los autores agradecen a la Universidad Nacional del Centro del Perú.

References

[1] Kreutz, D., Ramos, F.M.V., Veríssimo, P.E., Rothenberg, C.E., Azodolmolky, S., Uhlig, S., 2015. Software-defined networking: A comprehensive survey. Proceedings of the IEEE 103, 14–76. doi:10.1109/JPROC.2014.2371999.

[2] Kumar, S., Gupta, S., 2025. SDN TCP-SYN dataset: A dataset for TCP-SYN flood DDoS attack detection in software-defined networks. Data in Brief 59, 111314. doi:10.1016/j.dib.2025.111314.

[3] Kumar, S., Gupta, S., 2026. ICMP-Flood-SDN: A Python based machine learning application for ICMP flood DDoS attack detection in software defined networks. Software Impacts 27, 100819. doi:10.1016/j.simpa.2026.100819.

[4] Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116. doi:10.5220/0006639801080116.

[5] Sharafaldin, I., Lashkari, A.H., Hakak, S., Ghorbani, A.A., 2019. Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy, in: Proceedings of the 53rd IEEE International Carnahan Conference on Security Technology (ICCST), pp. 1–8. doi:10.1109/CCST.2019.8888419.

[6] Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A., 2009. A detailed analysis of the KDD CUP 99 data set, in: Proceedings of the 2nd IEEE International Conference on Computational Intelligence for Security and Defense Applications (CISDA).